Applying Radial Basis Function Neural Networks Based on Fuzzy Clustering to Estimate Web Applications Effort

Abdelali Zakrani[1], Ali Idri[1]

Abstract – *An important research issue in software project management is how to predict accurately an effort for the software project to develop at an early stage. Achieving high accuracy when estimating the effort of the software under development would supply effectively the project managers in leading to projects finished on time and within budget. Unfortunately, existing software effort estimation techniques are still unable to provide acceptable estimates. This paper investigates the application of Radial Basis Function Neural Networks (RBFN) based on fuzzy clustering in web development effort estimation. The proposed model is designed by integrating the principles of RBFN and the fuzzy C-means clustering algorithm. The architecture of the network is suitably modified at the hidden layer to realize a novel neural implementation of the fuzzy clustering algorithm. Fuzzy set-theoretic concepts are incorporated at the hidden layer, enabling the model to handle uncertain and imprecise data, which can significantly improve the accuracy of the obtained estimates. MMRE and Pred are used as measures of prediction accuracy for the undertaken study. The results show that an RBFN using fuzzy C-means performs better than an RBFN using hard K-means. This study uses data on web applications from the Tukutuku database.* **Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved.**

Keywords: *Web Effort Estimation, RBF Neural Networks, Fuzzy C-means, Tukutuku Dataset*

Nomenclature

ANN: Artificial Neural Network
MLP: Multi-Layer Perceptron
RBFN: Radial Basis Function Network
APC-III: extension of Adaptive Pattern Classifier APC-I
SLIM: Software Life Cycle Management
COCOMO: COnstructive COost MOdel
SLOC: Source Line Of Code
FPA: Function Points Analysis
MRE: Magnitude Relative Error
MMRE: Mean of Magnitude Relative Error
MARE: Mean Absolute Relative Error
MAPE: Mean Absolute Percentage Error
WNN: Wavelet Neural Network
ENNA: Ensemble of Neural networks with Associative Memory
FCM: Fuzzy C-means clustering algorithm
XB: Xie-Beni's fuzzy clustering validity index

I. Introduction

Accurate and reliable effort estimate of the software project under development is the most crucial information for the project managers. Having this vital information allows them to make more effective project planning, control and staffing, therefore leading to project delivered on time and within budget. Whereas, managing the software project without accurate estimate makes the task of the project manager more difficult and stressful. As a result, he/she may take wrong decisions threatening the success of the whole project. As case in point, when the software project is under-estimated, the manager probably will under-staff it (resulting in staff burnout), under-scope the quality assurance effort (running the risk of delivering poor quality product), and set too short schedule (resulting in loss of credibility as the deadlines are missed).

Aiming at assisting project managers in their tasks, numerous software development effort estimation techniques have been developed over the last three decades [1]. In a recent systematic review [2], Jorgensen and Shepperd identified 304 papers published in 76 journals dealing with the development and the improvement of software effort estimation approaches (more details can be found in [2]). These latter are based on different techniques varying from expert judgment [3] and statistical analysis [4] to artificial intelligence tools [5].

In recent years, Artificial Neural Networks (ANNs) have been emerged as successful modeling techniques when seeking an approximation of a nonlinear relationship between the independent and dependent variables. Previously, Hornik et al. [6] have proved that ANN are capable to model any measurable function. In the context of software effort estimation, artificial neural networks have been increasingly employed to model the

Manuscript received June 2010, revised July 2010
Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

complex relationship linking the software attributes and software development effort [7]-[8]-[9]. The most used architecture network in the aforementioned references is the multilayer perceptron (MLP). Unlike these latter, this paper uses Radial Basis Function (RBFN) neural networks to address estimation effort issue.

The use of Radial Basis Function Networks to predict software development effort requires the determination of some parameters. In an earlier work [10], we have empirically evaluated the use of some crisp clustering techniques in designing the middle layer of RBFN for software effort estimation. Particularly, the two used crisp clustering techniques were the K-means and the APC-III algorithms. The two studies were based on the COCOMO'81 and a web hypermedia dataset that contains 53 software projects. We have found that the RBFN designed with the K-means algorithm performs better, in terms of effort estimates accuracy, than the RBFN designed with APC-III algorithm for the two datasets.

The purpose of the present paper is to assess and to discuss the use of fuzzy clustering techniques, especially the fuzzy C-means algorithm in designing RBFN networks for software effort estimation. Instead of crisp (no fuzzy) clustering techniques where each software project belongs to exactly one cluster, fuzzy clustering techniques will allow to each software project to belong to more than one cluster with different degrees. Hence, the designed model may handle correctly the imprecision and the uncertainty when assigning a software project to clusters. This study uses 53 web projects from Tukutuku database [11].

The rest of this paper is structured as follows: Section 2 provides some background information about the effort estimation literature. It is followed in Section 3 by a brief description of RBF Networks. In Section 4, we describe how fuzzy logic is incorporated in the hidden layer of the RBFN networks by using fuzzy C-means. A description of historical projects dataset and evaluation criteria is presented in Section 5. Experiment design and results obtained are addressed in Section 6 and 7 respectively. In Section 8, the concluding remarks are stated and the future work is discussed.

II. Literature Review

Although the several software effort estimation models developed using a broad range of techniques and tools, accurate and reliable estimates are still crucial goal. Moreover, the problem has been shown to be a complex issue and the results obtained have not in general been encouraging enough.

II.1. Algorithmic Based Estimation Models

Putnam [12] proposed an early algorithmic model baptized SLIM model (Software Life cycle Management). In a nutshell, SLIM model is based on the assumption that development effort varies with time and this can be modeled with some degree of accuracy by Rayleigh function. The central part of Putnam's model is called software equation and is calculated as follows:

$$S_s = C \times K^{\frac{1}{3}} \times {t_d}^{\frac{4}{3}} \quad (1)$$

where S_s is the estimated size of the software system, K *is* the total lifecyle effort, C is the technology constant and t_d is the development time.

Boehm [4] developed one of the best known and the most widely used estimation models, the original COCOMO, called COCOMO'81. This model uses a basic regression formula (see Eq. 2), with parameters that are derived from historical project data and current project attributes. COCOMO'81 uses the software size as primary input effort driver. The model was revised and updated yielding to COCOMO II [13] which has integrated new effort drivers to accommodate better to modern language programming as well as software generation tools.

$$E = a \times S^b \quad (2)$$

(where E is the effort required, S is the estimated size and a and b are domains specific constants)

Both SLIM and COCOMO models are based on the estimated software size to derive an estimate for the effort, and yet it is difficult to predict accurately the required effort at an early stage. To address this issue, Abrecht developed *function point* method [14] which consists in the measure of the function that the software is to perform. The *function* is quantified as *function point*, essentially, a weighted sum of the number of inputs, outputs, master files, and inquiries provided, or generated by the software. Albrecht states that *function point* measure is thought to be more useful than SLOC as a prediction of effort because function points are relatively easily estimated from a statement of basic requirements for a program early in the development cycle [14].

Despite their popularity, algorithmic models suffer from serious drawbacks [15]. First of all, they are unable to capture the complex relationship between effort and software development environment since they are based on the estimated size (SLOC, FPA…) as the major determinant of effort. In addition, algorithmic models can only be calibrated manually. As a matter of fact, Kemerer [15] performed an empirical validation of the above algorithmic models using data from projects outside the original model development environments without recalibrating the models. The results indicate that the models developed in different environments do not work very well when they are uncalibrated, as might be expected. Average error rates calculated using the MRE formula ranged from 85 to 772 percent.

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

II.2. Machine Learning Based Estimation Models

The problems encountered when using algorithmic models have emphasized many researchers to look for new techniques to solve the software effort estimation issue. In this respect, machine learning methods have shown to be a promising alternative to the algorithmic models [16]. These methods focus on learning from the past projects to provide estimations for the future projects. In other words, they attempt to approximate the required effort without making any assumption about the form of the function linking the effort and the software attributes. The most machine learning approaches investigated in the literature are: case-based reasoning, decision trees, artificial neural networks, and genetic programming [2].

Case based reasoning is the process of identifying one or more historical projects that are similar to the project under development and then generating the estimates from them. Shepperd and Scofield developed Angel, a case based reasoning approach to estimate the software effort [17]. They validated the results in nine different datasets and showed that estimation by analogy produces a superior predictive performance in all used datasets when measured by MMRE and in seven out of nine datasets for the Pred(25) indicator. In [18], the authors propose a fuzzy version of case based reasoning to handle imprecise and uncertain data.

Decision tree is a flow-chart-like hierarchical tree structure which is composed of three basic elements: decision nodes corresponding to software project attributes, edges or branches which correspond to the different possible software attribute values. The third component is leaves including projects that typically belong to the same class or that are very similar. A decision tree can be built using one of the several induction algorithms developed [19]. Previously, Porter and Selby have used ID3 algorithm to induce a decision tree for estimating software development effort and other resource-related dimensions [20]-[21]. Srinivasan and Fisher experiment the use of CARTX to construct a decision tree [5]. The results showed possible advantages of this approach over traditional models. Other non-standard decision trees were proposed. Namely, fuzzy decision trees [22]-[23] which blend decision trees with fuzzy set tools to manage fuzzy information (attribute and class values are vaguely expressed with linguistic fuzzy terms).

Genetic programming is a variation on the genetic algorithms, in which the representation of the chromosomes is not a binary string but other data structures. For instance in [24], the software effort equations are treated as chromosomes. Initially, a number of random effort equations are generated. Afterwards, a new population of effort equations is created from previous effort equations by applying genetic operations to the most accurate equations until an accurate enough effort equation is reached. Several relative works indicate that genetic programming can offer significant improvements in the effort estimation accuracy [25]-[26]-[27].

Recently, artificial neural networks have attracted numerous researchers due to their approximation and learning abilities. In software effort estimation issue, they have become an important alternative approach to algorithmic models [28]. The first study concerned the use of ANNs to predict software development effort were undertaken by Hsu and Tenorio [29]. They used Multilayer Perceptron (MLP) constructed using self organizing neural network and backpropagation algorithms. Their preliminary results showed that ANN models have been superior in performance, learning time, modeling power, and require fewer prior assumptions than traditional software effort estimation models. In [5], the authors compared two approaches (1) a back propagation neural network and (2) regression trees, using COCOMO database. Their experimental results showed that neural network and regression trees are competitive with algorithmic based models. Moreover, using a dataset from Australian Software Metrics Association, ANN estimation models showed to be more accurate, in term of MARE, than CBR estimation models [30].

In [8], Heiat compared neural network estimation method to regression approach for software effort estimation. The results of his comparison indicate that neural network approach was competitive with regression when third generation language data set was used in the study. However, in his second study when a combined third and fourth generation language data set were used, the computed Mean Absolute Percentage Error (MAPE) of the neural network model was significantly smaller than MAPE of regression model.

Kumar al al. used wavelet neural network to forecast the software development effort [9]. Also, they have compared the effectiveness of their proposed WNN model to other types of neural network and traditional models. The study was based on Canadian Financial dataset and IBM data processing services dataset. Their results suggest that wavelet neural network outperformed all the other techniques in terms of mean magnitude relative error (MMRE).

However, Neural networks estimation models are complex with too many parameters to adjust. Hence, relatively large datasets are required for accurate and stable estimates. Furthermore, they are sensitive to training data, prone to get stuck in local minimums [5]-[31]. To overcome these drawbacks, Kultur et al. proposed the use of the ensemble of neural networks with associative memory (ENNA) [31]. In this latter model an ensemble is used rather than a single MLP. When a new project is given to the model for estimation, each MLP provides its own result. These results are clustered and the average of the largest cluster is returned. Thereafter, associative memory is used for bias correction. According to [31], ENNA are shown to be not only more

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

accurate than single MLP, but also they are stable and reliable.

III. RBF Neural Networks

The architecture of RBFN Neural Network is quite simple. It involves three different layers. An input layer which consists of sources nodes (effort drivers); a hidden layer in which each neuron computes its output using a radial basis function, which is in general a Gaussian function, and an output layer which builds a linear weighted sum of hidden neuron outputs and supplies the response of the network (effort). A RBF neural network configured for software effort estimation has only one output neuron. So, it implements the output-input relation in (Eq. 3) which is indeed a composition of the nonlinear mapping realized by the hidden layer with the linear mapping realized by the output layer

$$F(x) = \sum_{j=1}^{C} \beta_j \phi_j(x) \tag{3}$$

where C is the number of hidden neurons, $x \in \mathbb{R}^n$ is the input, β_j are the output layer weights of the RBFN networks and $\phi(x)$ is Gaussian radial basis function given by:

$$\phi_j(x) = e^{\left(-\frac{\|x-c_j\|^2}{\sigma_j^2}\right)} \tag{4}$$

where $c_j \in \mathbb{R}^n$ and σ_j are the center and the width of jth hidden neuron respectively and $\|.\|$ denotes the Euclidean distance.

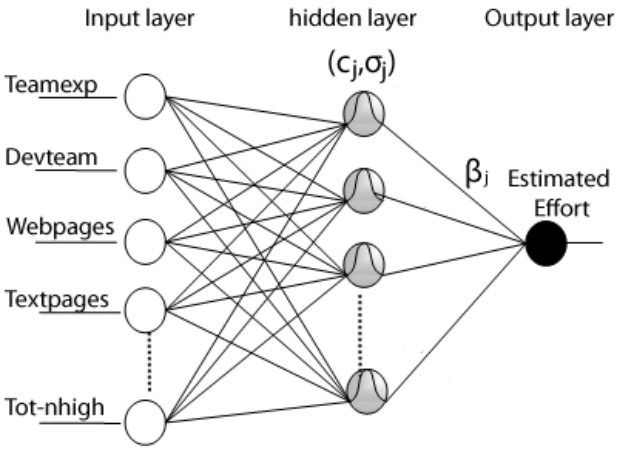

Fig. 1. A Radial Basis Function Network architecture for web development effort estimation.

IV. Fuzzy Clustering in the Design of RBF Neural Networks

Fuzzy C-means (FCM) algorithm, also known as Fuzzy ISODATA, was introduced by Bezdek [32], as an extension to Dun's Algorithm [33]. The FCM-based algorithms are the most widely used fuzzy clustering algorithms in practice.

Let $X = \{x_1, x_2, \cdots, x_N\}$, where $x_i \in \mathbb{R}^n$ present a given data set of historical projects. The objective of FCM-algorithm is to minimize the fuzzy C-means cost function formulated as

$$J(U,V) = \sum_{j=1}^{C} \sum_{i=1}^{N} (u_{ij})^m \|x_i - v_j\|^2 \tag{5}$$

$V = \{v_1, v_2, \cdots, v_C\}$ are the cluster centers. $U = (u_{ij})_{N\times C}$ is a fuzzy partition matrix, in which each u_{ij} indicates the degree of membership between the project x_1 and the cluster j. the values of the matrix U should satisfy the following conditions

$$u_{ij} \in [0,1] \quad \forall\, i = 1, \cdots, N \quad \forall\, j = 1, \cdots, C \tag{6}$$

$$\sum_{j=1}^{C} u_{ij} = 1, \quad \forall\, i = 1, \cdots, N \tag{7}$$

The exponent $m \in [1, \infty]$ is a weighting factor which determines the fuzziness of the clusters and usually chosen in the range of $[1.5, 2.5]$, in our experiments we use $m = 1.5$. The most commonly used distance norm is the Euclidean distance $d_{ij} = \|x_i - v_j\|$.

Minimization of the cost function $J(U,V)$ is nonlinear optimization problem which can be minimized with the following iterative algorithm.

Step 1: Initialize the membership matrix U with random values so that the conditions (Eq.6) and (Eq.7) are satisfied. Choose the appropriate exponent m and the termination criteria ε.

Step 2: Calculate the cluster centers V according to the equation:

$$v_j = \frac{\sum_{i=1}^{N} (u_{ij})^m x_i}{\sum_{i=1}^{N} (u_{ij})^m} \qquad \forall\, j = 1, \cdots, C \tag{8}$$

Step 3: Calculate the new distance norms:

$$d_{ij} = \|x_i - v_j\| \quad \forall\, i = 1, \cdots, N \quad \forall\, j = 1, \cdots, C \tag{9}$$

Step 4: Update the fuzzy partition matrix U:
If $d_{ij} > 0$ (indicating that $x_j \neq v_j$)

$$u_{ij} = \frac{1}{\sum_{k=1}^{C} \left(\frac{d_{ij}}{d_{ik}}\right)^{\frac{2}{m-1}}} \tag{10}$$

Step 5: If the termination criterion has been met, stop
Else go to **Step 2**

A suitable termination criterion could be to evaluate the cost function (Eq. 5) and to see whether it is below a certain tolerance value or if its improvement compared to

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

the previous iteration is below a certain threshold. Also the maximum number of iteration cycles can be used as a termination criterion.

V. Data Description and Evaluation Criteria

This section describes the dataset used to perform the empirical study, and presents also the evaluation criteria adopted to compare the estimating capability of the developed models.

V.1. Data Description

The data used in the present study come from collection of historical web applications project namely Tukutuku dataset.

The Tukutuku dataset contains 53 Web projects [11]. Each Web application is described using 9 numerical attributes such as: the number of html or shtml files used, the number of media files and team experience (see Table 1). However, each project volunteered to the Tukutuku database was initially characterized using more than 9 software attributes, but some of them were grouped together. For example, we grouped together the following three attributes: number of new Web pages developed by the team, number of Web pages provided by the customer and the number of Web pages developed by a third party (outsourced) in one attribute reflecting the total number of Web pages in the application (Webpages). By analyzing statistically Tukutuku dataset, we noticed that the data is skewed and containing several outlier projects (Table 2).

TABLE I
SOFTWARE ATTRIBUTES FOR TUKUTUKU DATASET AND ITS DESCRIPTION

Software Attribute	Description
Teamexp	Average number of years of experience the team has on Web development
Devteam	Number of people who worked on the software project
Webpages	Number of web pages in the application
Textpages	Number of text pages in the application (text page has 600 words)
Img	Number of images in the application
Anim	Number of animations in the application
Audio/video	Number of audio/video files in the application
Tot-high	Number of high effort features in the application
Tot-nhigh	Number of low effort features in the application

TABLE II
DESCRIPTIVE STATISTICS FOR TUKUTUKU DATASET

Software Attribute	Min	Max	Mean	Median	Standard deviation
Teamexp	1	10	4,30	5	2,01
Devteam	1	8	2,58	2	1,55
Webpages	3	2000	155,04	36	341,11
Textpages	0	2190	77,92	9	313,97
Img	0	1820	184,60	75	350,40
Anim	0	170	11,57	0	33,05
Audio/video	0	4	0,17	0	0,64
Tot-high	0	12	2,30	1	2,97
Tot-nhigh	0	16	5,34	5	3,97
Tot-Effort	6	5000	414,85	105	825,40

V.2. Evaluation Criteria

We employ the following criteria to assess and compare the accuracy of the effort estimation models. A common criterion for the evaluation of effort estimation models is magnitude of relative error (MRE), which is defined as

$$MRE = \left|\left(\frac{Effort_{actual} - Effort_{estimated}}{Effort_{actual}}\right)\right| \quad (11)$$

The MRE values are calculated for each project in the dataset, while mean magnitude of relative error (MMRE) computes the average over N projects

$$MMRE = \frac{1}{N}\sum_{i=1}^{N} MRE_i \quad (12)$$

Generally, the acceptable target value for MMRE is 25%. This indicates that on the average, the accuracy of the established estimation models would be less than 25%.

Another widely used criterion is the Pred(*l*) which represents the percentage of MRE that is less than or equal to the value *l* among all projects. This measure is often used in the literature and is the proportion of the projects for given level of accuracy. The definition of Pred(*l*) is given as follows:

$$Pred(l) = \frac{k}{N} \quad (13)$$

Where *N* is the total number of observations and *k* is the number of observations whose MRE is less or equal to *l*.
A common value for *l* is 0.25, which also used in the present study. The Pred(0.25) represents the percentage of projects whose MRE is less or equal to 25%. The Pred(0.25) value identifies the effort estimates that are generally accurate whereas the MMRE is fairly conservative with a bias against overestimates.

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

VI. Experiment Design

As mentioned earlier, the use of an RBFN to estimate software development effort requires the determination of its architecture parameters according to the characteristics of the web Applications dataset, especially the number of input neurons, number of hidden neurons, centers c_i, widths σ_j and weights β_j.

The number of input neurons is the same as the number of attributes (effort drivers) describing the historical software projects in the web applications dataset. Hence, the number of input neurons is equal to 9. The number of hidden neurons is determined by the number of clusters (C) provided by the hard or the fuzzy C-means algorithms described in Section 4. Concerning the widths σ_j, typically two alternatives are considered. The first one consists in taking the widths σ_j equal to a constant for all Gaussian functions [34]-[35]. For example in [35], the widths are fixed as follows:

$$\sigma = \frac{d_{max}}{\sqrt{2C}} \qquad (14)$$

where C is the number of centers and d_{max} is the maximum distance between those centers. Such a procedure fixes the degree of overlapping of the Gaussian kernels. It allows finding a compromise between locality and smoothness of the functions $\phi_j(x)$. This choice would be close to optimal solution if the data were uniformly distributed in the input space, leading to uniform distribution of the centroids. Unfortunately, in our data set, it is not the case, and using the formula above may lead to small or too flat values of σ_j which can easily influence the accuracy of estimates (see section 7). To avoid this problem we propose to determine a width factor, common to all Gaussian kernels. The widths of the kernels are then defined as:

$$\forall\, j, \qquad \sigma = r \times \frac{d_{max}}{\sqrt{2C}} \qquad (15)$$

By inserting the widths scaling factor, r, the approximation of the function $\phi_j(x)$ is smoothed such that the generation process is more efficient, as we allow an optimal overlapping of the Gaussian of the kernels.

The second option consists in estimating the width of each Gaussian function independently. For example, in [34] the width is set as follows:

$$\sigma_j = r \times \left\| c_j - c_{j_nearest} \right\|, \qquad j = 1, \cdots, C \qquad (16)$$

where $c_{j_nearest}$ is the nearest neighbor of the center c_j and r is an overlap factor determined empirically.

We notice that for RBFN with hard K-means we keep the same design described [10] and which have given us the best possible results in the terms of estimates accuracy.

Concerning the weights β_j, we may set each β_j to the associated effort of the center of the jth neuron. However, this technique is not optimal and does not take into account the overlapping that may exist between receptive fields of the hidden layer. Thus, we use the Delta rule to derive the values of β_j.

VII. Results of the Experimental Studies

The following section presents and discusses the results obtained when applying the RBFN to the web dataset. The calculations were made using two software prototypes developed with C language under a Microsoft Windows PC environment. The first software prototype implements the hard and the fuzzy C-means clustering algorithms, providing both the clusters and their centers from the web applications dataset. The second software prototype implements a effort estimation model based on an RBFN architecture in which the middle-layer parameters are determined by means of the first software prototype.

We have conducted several experiments with both the hard K-means and the fuzzy C-means algorithms to decide on the number of hidden units. These experiments use the full web applications dataset for training. Choosing the 'best' classification to determine the number of hidden neurons and their centers is a complex task. For software effort estimation, we suggest that the 'best' classification is the one that satisfies the following two criteria:

- It provides coherent clusters, i.e. the software projects of a given cluster have satisfactory degrees of similarity;
- It improves the accuracy of estimates, in terms of MMRE and Pred(0.25), produced by the RBFN .

To measure the coherence of clusters in the case of fuzzy C-means algorithm, we use the well known fuzzy clustering validity function developed by Xie and Beni [36] and defined as follow:

$$XB = \frac{\sum_{i=1}^{C}\sum_{k=1}^{N}\left(u_{ij}\right)^2 \|c_i - x_k\|^2}{N \times \min_{i,k}\|c_i - c_k\|} \qquad (17)$$

A small value of XB means a more compact and separate clustering. The goal should therefore be to minimize the value of XB.

VII.1. Study 1: Impact of the Widths on Accuracy of RBFN Estimates

The objective of this first study is to illustrate the impact of the widths on the accuracy of estimates

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

produced by RBF network. So, we have used a design of a RBF Network with 25 neurons in the hidden layer, and we have conducted several experiments each time using a different value for the width. The results of this first study are shown in Fig. 2. As it can be seen, too small and too flat values of σ lead to poor accuracy estimates (when $\sigma < 35$ and $\sigma > 50$). Whereas, when a width value is between 35 and 50, it yields to better accuracy estimates and exactly when $\sigma = 40$ (MMRE=38.03 and Pred(0.25)=71.10).

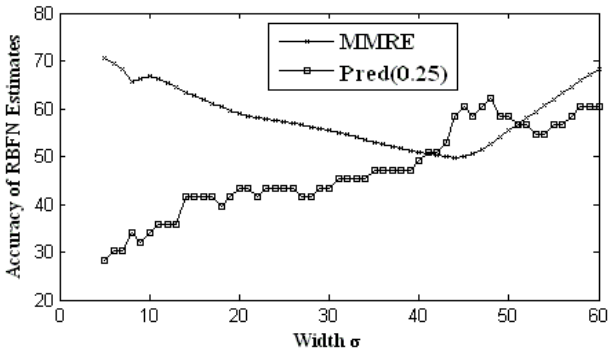

Fig. 2. Variation of the accuracy measures, MMRE and Pred, according to width values.

We have further analyzed whether the above results are sensitive to the number of neurons in the hidden layer of the RBF network. In other words, we have trained and tested RBF network with different number of hidden neurons and with three values of σ (10,40,50). the results obtained are consistent with the results of the previous experiment (figure 3 and 4).

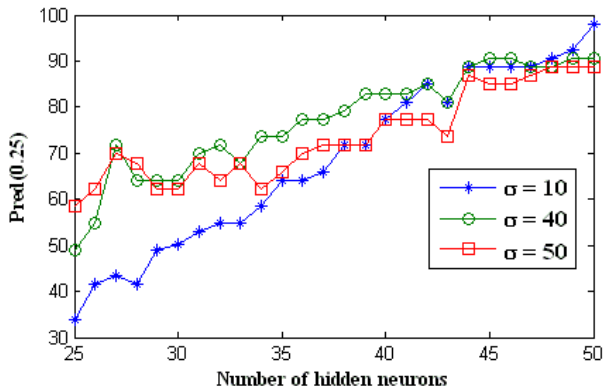

Fig. 3. Variation of Pred(0.25) measure according to the number of hidden neurons.

VII.2. Study 2: Comparison Between RBFN-Fuzzy C-means and RBFN-K-means

The primary goal of the second study is to evaluate the impact of the incorporation of fuzzy logic in the construction of the hidden layer of RBF network by means of fuzzy C-means algorithm. Also, the study aims to examine the role of the formula of width discussed in

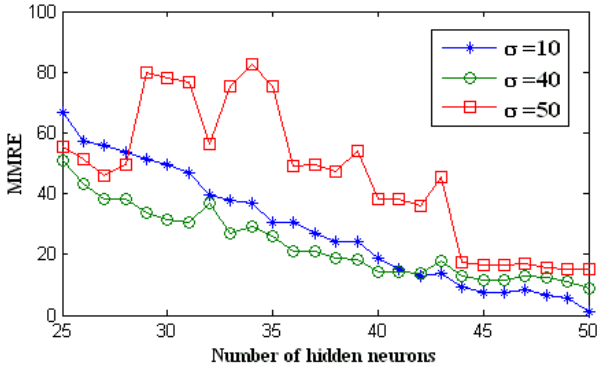

Fig. 4. Variation of MMRE measure according to the number of hidden neurons.

the last section. Hence, we have conducted a comparison between the three following RBF network models:

Model 1: *RBFN-FCM- Sigma-Saha*, this model is designed using fuzzy C-means algorithm and Saha's width formula to determine (c_j, σ_j).

Model 2: *RBFN-FCM- Sigma-Haykin*, this model is designed using fuzzy C-means algorithm and the modified Haykin's width formula to determine (c_j, σ_j).

Model 3: *RBFN-KM*, this model is designed using K-means algorithm to determine c_j and a formula of width developed in a previous work [10].

In order to improve the estimates accuracy obtained by RBFN, we need to optimize the widths of the Gaussian kernels. For this issue we have conducted several experiments with an RBFN, each time using one the formula given in section 6 to determine *width factor r* which leads to the best estimates. Therefore, the best accuracy estimation was achieved for the ***model 1*** when *r=0.5* and for the ***model 2*** when *r=0.05*. Thus, we have used these values of width factor in our following experiments.

From each one of the designed models, different configurations have been obtained by varying the number of hidden neurons *C*. In fact, this will allow us to assess whether these three models are sensitive to the number of hidden neurons. Besides, this procedure will allow us the find the optimal RBF network (model and size). For this reason, we have trained and tested each model using Tukutuku dataset. The learning rate was set to 0.03 for the all experiments. The results obtained for a number of hidden neurons equal to 30 are provided in table III. As it can be seen from these results, the RBF network models based on fuzzy C-means perform better than that based on K-means. And the *model 2* outperforms the *model 1* in terms of Pred(0.25). Though, it made a higher MMRE than *model 1*. However, as MMRE measure is extremely sensitive to individual predictions with excessively large MREs, we can consider that the *model 2* is more accurate and reliable than the *model 1*.

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

TABLE III
RESULTS OF COMPARATIVE ANALYSIS

Model	Number of hidden neurons	MMRE	Pred(0.25)
Model 1: RBFN-FCM with Sigma-Saha	30	34.04	64.15
Model 2: RBFN-FCM with Sigma-Haykin	30	37.34	71.70
Model 3: RBFN- K-means	30	79.81	52.83

Fig 4: Variation of Pred(0.25) measure according to the number of hidden neurons.

Fig. 5 shows and compares the complete results of the three previous models when varying the number of hidden neurons from 25 to 52. From this figure we note that the RBF network models based on fuzzy C-means, *model 1* and *model 2*, perform much better in terms of MMRE than that based on K-means clustering, *model 3*. As case in point, for C=29 the *model 3* made a much higher prediction error (MMRE=79.81) than the model 2 (MMRE= 34.04) and *model 3* (MMRE= 37.34). We can notice that when C is lower than 35, the *model 3* gives, in general, poorer estimates, which can interpret by the fact that the classifications obtained for C lower than 35 are less coherent, i.e contain some clusters which are composed of software projects that are not sufficiently similar.

Fig. 6 compares the accuracy of the three above models, in terms of Pred(0.25), when varying the number of hidden neurons. As can be seen, the accuracy of RBF network based on fuzzy C-means, *model 1* and *model 2*, perform much better than that based on K-means algorithm. Indeed, the *model 3* generates poor estimates even when increasing the number of hidden neurons until 33. Whereas the *model 2* produced acceptable effort estimates with a number of hidden neurons C=27.

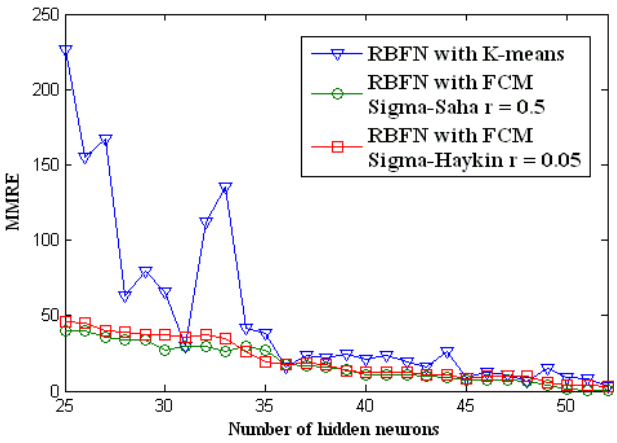

Fig 5: Variation of MMRE measure according to the used clustering algorithm.

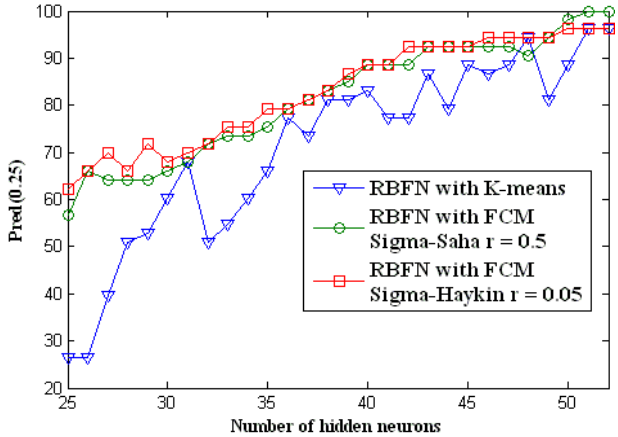

Fig. 6. Variation of Pred(0.25) measure according to the used clustering algorithm.

VIII. Conclusion and Future Work

In this paper, we have empirically studied the use of two clustering techniques when designing RBF neural network for software effort estimation. The two used clustering algorithms are the well-known hard and the fuzzy C-means. This study is based on one dataset containing 53 web applications. We used the entire dataset to train and test the designed RBFN. We have found that the RBFN designed with the fuzzy C-means algorithm performs better, in terms of the effort estimates accuracy, than the RBFN designed with the hard K-means algorithm. To confirm this result, we are looking currently in applying cross validation methods to compare the generalization ability of hard and fuzzy C-means based RBFN for software development effort estimation.

References

[1] L. C. Briand and I. Wieczorek, "Resources Estimation in Software Engineering," *Encyclopedia of Software engineering,* pp. 1160-1196, 2002.

[2] M. Jørgensen and M. Shepperd, "A Systematic Review of Software Development Cost Estimation Studies," *IEEE Transactions on Software Engineering,* vol. 33, pp. 33-53, 2007.

[3] R. T. Hughes, "Expert Judgement as an Estimating Method," *Information and Software Technology,* vol. 38, pp. 67-75, 1996.

[4] B. W. Boehm, *Software Engineering Economics*: Prentice Hall PTR, 1981.

[5] K. Srinivasan and D. Fisher, "Machine Learning Approaches to Estimating Software Development Effort," *IEEE Trans. Software Eng.,* vol. 21, pp. 126-137, 1995.

[6] K. Hornik, M. Stinchcombe, and H. White, "Multilayer Feedforward Networks Are Universal Approximators," *Neural Networks,* vol. 2, pp. 359-366, 1989.

[7] G. E. Wittig and G. R. Finnie, "Estimating Software Development Effort with Connectionist Models," *Information and Software Technology,* vol. 39, pp. 469-476, 1997.

[8] A. Heiat, "Comparison of Artificial Neural Network and Regression Models for Estimating Software Development Effort," *Information and Software Technology,* vol. 44, 2002.

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

[9] K. Vinay Kumar, V. Ravi, M. Carr, and N. Raj Kiran, "Software Development Cost Estimation Using Wavelet Neural Networks," *Journal of Systems and Software,* vol. 81, pp. 1853-1867, 2008.
[10] A. Idri, A. Zahi, E. Mendes, and A. Zakrani, "Software Cost Estimation Models Using Radial Basis Function Neural Networks," in *Software Process and Product Measurement,* vol. 4895, 2008, pp. 21-31.
[11] B. A. Kitchenham and E. Mendes, "A Comparison of Cross-Company and within-Company Effort Estimation Models for Web Applications," in *Metrics'04, Chicago, Illinois September 11-17 th 2004, IEEE Computer Society*, 2004, pp. 348-357.
[12] L. H. Putnam, "A General Empirical Solution to the Macro Software Sizing and Estimating Problem," *Software Engineering, IEEE Transactions on,* vol. SE-4, pp. 345-361, 1978.
[13] B. Boehm and C. A. and... *Software Cost Estimation with Cocomo Ii*: Prentice-Hall, 2000.
[14] A. J. Albrecht and J. E. Gaffney, Jr., "Software Function, Source Lines of Code, and Development Effort Prediction: A Software Science Validation," *Software Engineering, IEEE Transactions on,* vol. SE-9, pp. 639-648, 1983.
[15] C. F. Kemerer, "Empirical Validation of Software Cost Estimation Models," *Communications of the ACM,* vol. 30, pp. 416-429, 1987.
[16] M. O. Saliu and M. Ahmed, *Soft Computing Based Effort Prediction Systems - a Survey*, E. Damiani, L. C. Jain, and M. Madravio ed.: Springer-Verlag, July 2004.
[17] M. J. Shepperd and C. Schofield, "Estimating Software Project Effort Using Analogies," *IEEE Trans. Software Eng.,* vol. 23, pp. 736-743, 1997.
[18] A. Idri, A. Abran, and T. Khoshgoftaar, "Fuzzy Case-Based Reasoning Models for Software Cost Estimation," pp. 64-96, 2004.
[19] J. R. Quinlan, *C4.5: Programs for Machine Learning*: Morgan Kaufmann Publishers Inc., 1993.
[20] R. W. Selby and A. A. Porter, "Learning from Examples: Generation and Evaluation of Decision Trees for Software Resource Analysis," *Software Engineering, IEEE Transactions on,* vol. 14, pp. 1743-1757, 1988.
[21] A. A. Porter and R. W. Selby, "Empirically Guided Software Development Using Metric-Based Classification Trees," *Software, IEEE,* vol. 7, pp. 46-54, 1990.
[22] A. S. Andreou and E. Papatheocharous, "Software Cost Estimation Using Fuzzy Decision Trees," L'Aquila, 2008, pp. 371-374.
[23] A. Idri and S. Elyassami, "Software Cost Estimation Using Decision Trees," in *International Symposium on INnovations in Intelligent SysTems and Applications*, Kayseri & Cappadocia, Turkey, 2010.
[24] J. J. Dolado, "On the Problem of the Software Cost Function," *Information and Software Technology,* vol. 43, pp. 61-72, 2001.
[25] C. J. Burgess and M. Lefley, "Can Genetic Programming Improve Software Effort Estimation? A Comparative Evaluation," *Information and Software Technology,* vol. 43, pp. 863-873, 2001.
[26] Y. Shan, R. I. McKay, C. J. Lokan, and D. L. Essam, "Software Project Effort Estimation Using Genetic Programming," in *Communications, Circuits and Systems and West Sino Expositions, IEEE 2002 International Conference on*, 2002, vol. 2, pp. 1108-1112 vol.1102.
[27] M. Lefley and M. J. Shepperd, "Using Genetic Programming to Improve Software Effort Estimation Based on General Data Sets," in *Proceedings of the 2003 international conference on Genetic and evolutionary computation: PartII*, Chicago, IL, USA, 2003, pp. 2477-2487.
[28] A. R. Gray and S. G. MacDonell, "A Comparison of Techniques for Developing Predictive Models of Software Metrics," *Information and Software Technology,* vol. 39, pp. 425-437, 1997.
[29] W. Hsu and M. F. Tenorio, "Software Engineering Effort Models Using Neural Networks," in *Neural Networks, 1991. 1991 IEEE International Joint Conference on*, 1991, pp. 1190-1195 vol.1192.
[30] G. R. Finnie and G. E. Wittig, "Ai Tools for Software Development Effort Estimation," in *Software Engineering: Education and Practice, 1996. Proceedings. International Conference*, 1996, pp. 346-353.
[31] Y. Kultur, B. Turhan, and A. B. Bener, "Ensemble of Neural Networks with Associative Memory (Enna) for Estimating Software Development Costs," *Knowl.-Based Syst.,* vol. 22, pp. 395-402, 2009.
[32] J. C. Bezdek, *Pattern Recognition with Fuzzy Objective Function Algoritms*: Plenum Press, New York, 1981.
[33] J. C. Dunn, "Well Separated Clusters and Optimal Fuzzy-Partitions," *Journal of Cybernetics,* vol. 4, pp. 95-104, 1974.
[34] A. Saha and J. D. Keeler, "Algorithms for Better Representation and Faster Learning in Radial Basis Function Networks," in *Advances in Neural Information Processing Systems 2, NIPS*, 1989, pp. 482-489.
[35] S. Haykin, "Neural Networks: A Comprehensive Foundation," Prentice Hall, 1998.
[36] X. L. Xie and G. Beni, "A Validity Measure for Fuzzy Clustering," *IEEE Trans. Pattern Anal. Mach. Intell.,* vol. 13, pp. 841-847, 1991.

Authors' information

[1]Department of Software Engineering,
SI2M Laboratory, ENSIAS,
Mohammed Vth –Souissi University,
BP. 713, Madinat Al Irfane, Rabat, Morocco
Email: zakrani@gmail.com, idri@ensias.ma

Abdelali Zakrani received the B.Sc. degree in Computer Science from Hassan II University, Casablanca, Morocco, in 2003, and his DESA degree (M.Sc.) in the same major from University Mohammed V, Rabat, in 2005. Currently, he is preparing his Ph.D. in computer science in ENSIAS. His research interests include software cost estimation, software metrics, fuzzy logic, neural networks, decision trees.

Ali Idri is a Professor at Computer Science and Systems Analysis School (ENSIAS, Rabat, Morocco). He received DEA (Master) (1994) and Doctorate of 3rd Cycle (1997) degrees in Computer Science, both from the University Mohamed V of Rabat. He has received his Ph.D. (2003) in Cognitive Computer Sciences from ETS, University of Quebec at Montreal. His research interests include software cost estimation, software metrics, fuzzy logic, neural networks, genetic algorithms and information sciences.

Copyright © 2010 Praise Worthy Prize S.r.l. - All rights reserved

View publication stats